



S. B. JAIN INSTITUTE OF TECHNOLOGY, MANAGEMENT & RESEARCH, NAGPUR.

Post-Lab

Aim: The primary aim of this project is to develop a real-time, web-based American Sign Language (ASL) alphabet detector using machine learning. It bridges communication gaps by translating live hand gestures into readable text and audible speech directly within the user's browser.

Name of Student: Yash Raju Meshram

Roll No.:CM24D004

Semester/Year:VI/III

Academic Session:2025-26

Date of Performance:

Date of Submission:

Aim: The primary aim of this project is to develop a real-time, web-based American Sign Language (ASL) alphabet detector using machine learning. It bridges communication gaps by translating live hand gestures into readable text and audible speech directly within the user's browser.

Objectives

- **Real-Time Gesture Recognition:** To capture and process continuous live video feeds from a user's webcam to identify A–Z ASL alphabetic signs.
- **Static Image Analysis:** To provide a fallback/alternative feature allowing users to upload static images of signs for isolated prediction.
- **High-Confidence Filtering:** To implement a strict confidence threshold ($\geq 60\%$) to prevent false positives and ensure accurate translation.
- **Frame Stabilization:** To develop an algorithm that requires a sign to be held stably for a set number of consecutive frames (15 frames) before automatically appending it to the word builder, reducing erratic inputs.
- **Word Construction & Text-to-Speech (TTS):** To enable users to string individual letters together into complete words and sentences, and utilize the Browser Speech API to vocalize the constructed text.
- **Accessibility & UX:** To create an intuitive, responsive user interface featuring an active alphabet reference grid, sentence history tracking, and clipboard copying functionalities.
- **Client-Side Processing:** To deploy the machine learning model directly in the browser using TensorFlow.js, ensuring user privacy by eliminating the need to send camera feeds to a remote server.

Theory

3.1 American Sign Language (ASL)

American Sign Language is a complete, natural language that has the same linguistic properties as spoken languages, with grammar that differs from English. While ASL relies heavily on facial expressions and body movements, it also employs a manual alphabet (fingerspelling) for spelling out names, specific terms, or words that do not have a dedicated sign. This project focuses specifically on recognizing this manual alphabet (A–Z).

3.2 Convolutional Neural Networks (CNNs) and Image Classification

The core of the gesture recognition relies on Image Classification, a subset of computer vision. The model uses Convolutional Neural Networks (CNNs), which are deep learning algorithms specifically designed to process pixel data. CNNs apply various filters (kernels) to images to extract features like edges, textures, and eventually complex shapes (like the position of fingers in a hand sign).

To classify an image into one of the 26 alphabet classes, the final layer of the neural network typically employs a Softmax activation function.

3.4 TensorFlow.js (@tensorflow/tfjs)

TensorFlow.js is a library for machine learning in JavaScript. It allows the browser to run the pre-trained Teachable Machine model entirely client-side. This architecture provides zero-latency inference since the video frames do not need to be transmitted over a network to a backend server.

3.5 Web Speech API

To facilitate two-way communication, the system uses the native Web Speech API (SpeechSynthesis). This interface allows web applications to read out text content using the operating system's built-in voice synthesizers, granting a "voice" to the signs detected by the application.

4. Implementation

The implementation of SignSense is divided into two major phases: Model Training and Web Application Development.

4.1 Data Collection and Model Training

Platform Used: Google Teachable Machine (Image Project).

Dataset Construction: 26 distinct classes were created, representing letters A through Z. For each class, hundreds of sample images were captured using a webcam. Variations in hand distance, slight angle changes, and background variations were introduced to make the model robust.

Training Phase: The model was trained using the default MobileNet architecture provided by Teachable Machine.

Export: The model was exported as a TensorFlow.js format, yielding three specific files: model.json (the architecture), weights.bin (the learned parameters), and metadata.json (the class labels). These were placed in the local ./my_model/ directory.

4.2 Web Application Architecture

The application is built using standard web technologies:

HTML5: Structures the layout into a left column (Camera/Upload feed, Alphabet Reference) and a right column (Live Detection metrics, Word Builder, History).

CSS3: Provides a modern, responsive UI using variables, flexbox/grid layouts, and visual indicators (like the live recording dot and confidence progress bar).

JavaScript (ES6): Handles the application logic, DOM manipulation, and asynchronous model predictions.

4.3 Core Algorithmic Logic (app.js)

The logic operates via a continuous Request Animation Frame (RAF) loop:

Initialization (startCamera): Prompts the user for webcam access via `tmlImage.Webcam`. Upon approval, the video stream is piped into an HTML5 Canvas.

Prediction Loop (`cameraLoop` & `runPrediction`): The canvas frame is passed to `model.predict(src)`. The model returns an array of objects containing class names and probabilities. The algorithm sorts this to find the highest probability.

Confidence Thresholding: A strict constant `CONFIDENCE_THRESHOLD = 0.60` ensures that the system only reacts if the model is at least 60% sure of the gesture. If the top probability is below this, the UI displays "Waiting..." and a question mark.

Frame Stabilization (`stableFrames`): To prevent the word builder from being spammed with letters while the user is moving their hand into position, a stabilization algorithm is implemented.

The variable `STABILIZATION_THRESHOLD` is set to 15 frames.

If the system detects the *same* confident letter for 15 consecutive frames (roughly 0.5 seconds), it auto-appends that letter to the `wordBuffer`.

A locking mechanism (`canAddNext = false`) is engaged so the letter isn't added multiple times. The lock is only released when the user drops their hand or changes the sign.

4.4 Word Builder and Utility Integration

String Manipulation: Functions like `addSpace()`, `deleteLast()`, and `clearAll()` manipulate the `wordBuffer` string and instantly update the DOM.

Sentence Preservation: The `saveSentence()` function pushes the current string into an array (`sentenceHistory`), along with a timestamp, rendering it in the UI for later reference.

Text-to-Speech: The `speakText()` function utilizes `SpeechSynthesisUtterance`. It dynamically adjusts pitch and rate (set to 0.88 for clarity) to dictate the constructed word/sentence.

Output and Results

Figure 1: Application Interface (Idle State)

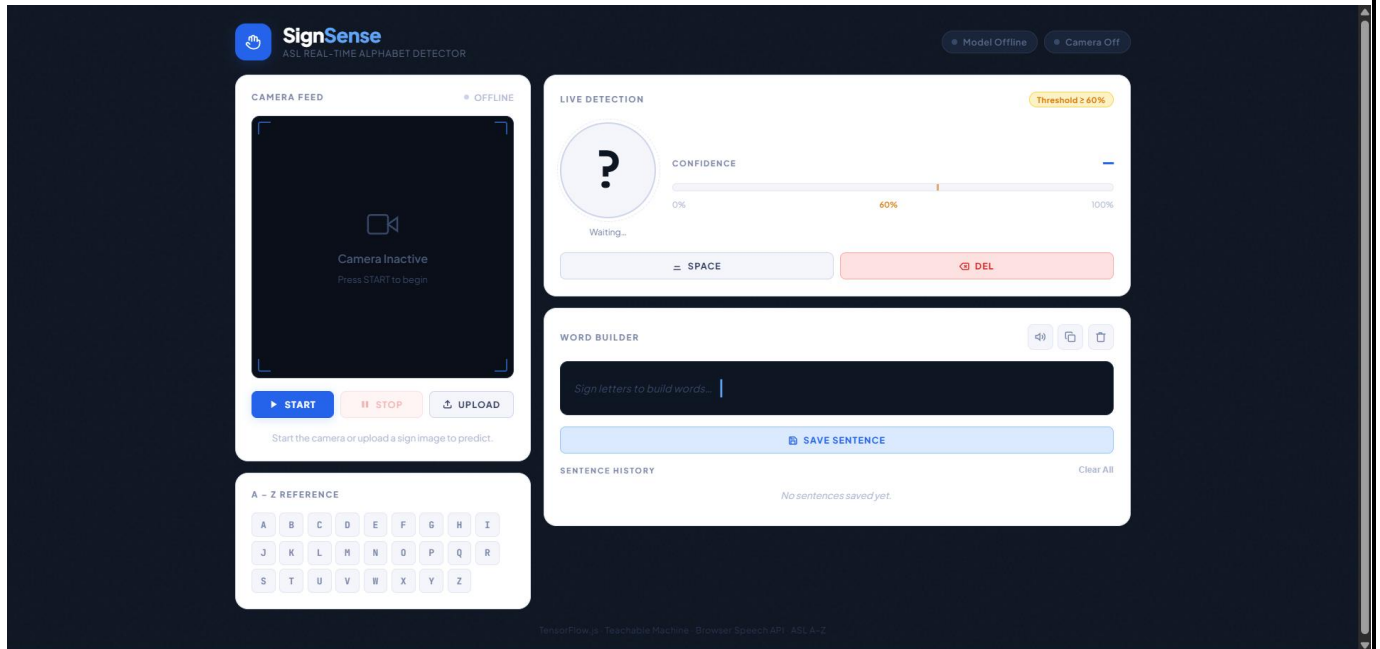


Figure 2: Real-Time Detection and Alphabet Highlighting

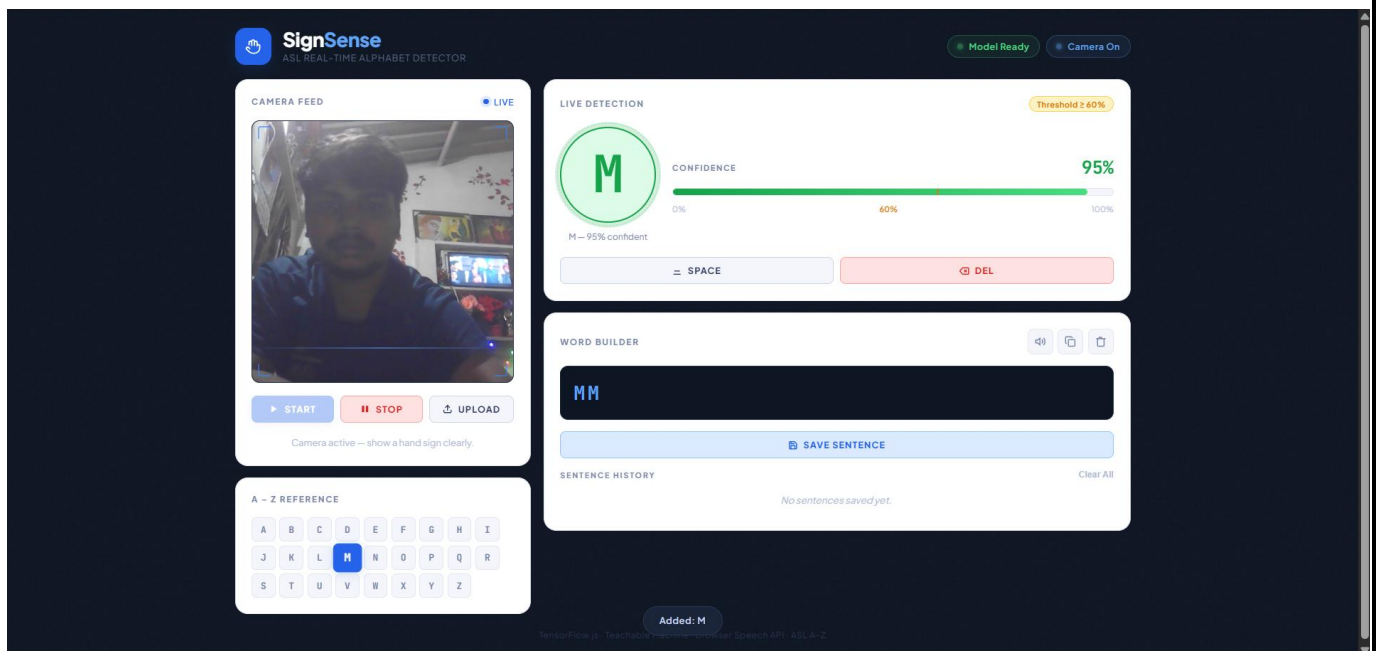
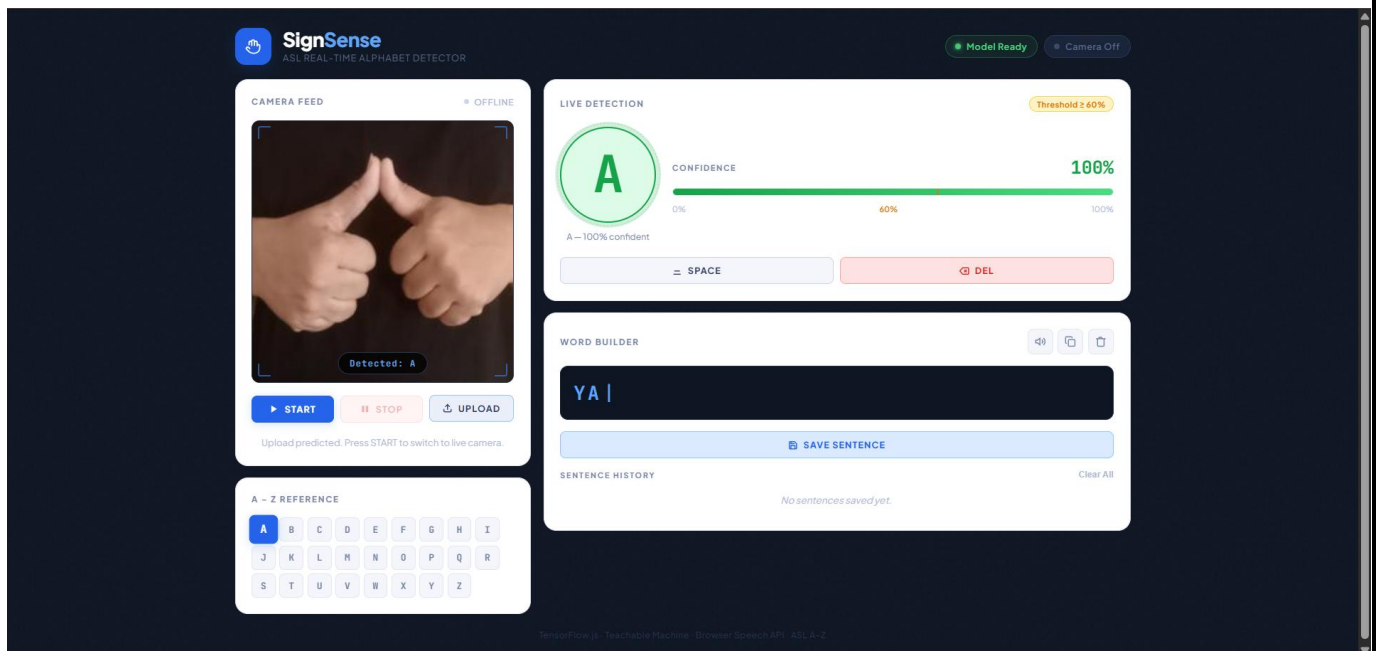


Figure 3: Image Upload Functionality



6. Result

The SignSense application successfully fulfills its objectives, functioning as a highly responsive real-time ASL detector.

Performance Metrics observed during testing:

- **Latency:** Because the TensorFlow.js model runs entirely client-side, inference times are practically instantaneous, typically processing at 30+ frames per second depending on the user's hardware.
- **Accuracy:** Under well-lit conditions with a clear contrast between the hand and the background, the model accurately predicts letters with confidence levels often exceeding 90%.
- **Stabilization Success:** The 15-frame stabilization threshold proved highly effective. It successfully filtered out transitional hand movements (blur) and only registered intentional, held signs, making the word-building process smooth and user-friendly.

Limitations Identified:

- **Background Interference:** Highly cluttered backgrounds or poor lighting conditions can cause the confidence score to drop below the 60% threshold, resulting in missed detections.
- **Dynamic Gestures:** Letters 'J' and 'Z' in ASL require motion. Because this model relies on static image classification, recognizing the motion-based letters requires the user to hold the final pose of the gesture, which slightly deviates from natural ASL signing.

6. Conclusion

The SignSense project demonstrates the powerful intersection of accessible web development and machine learning. By successfully integrating Google Teachable Machine models with TensorFlow.js, the application achieves real-time, privacy-respecting sign language detection without the need for expensive hardware or backend server infrastructure.

The addition of the stabilized word builder and the Web Speech API elevates the project from a simple machine learning classifier into a practical communication tool. It allows non-verbal individuals or ASL learners to spell out thoughts and have them spoken aloud seamlessly.